AMPLIACIÓN: Diseñar una página PHP con un autómata de estado-transición

Caso práctico

Carlos está haciendo el ejercicio de la media de una secuencia de números: primero se pide cuántos números hay, después se van pidiendo uno a uno y al final se muestra la media. Lleva una hora añadiendo `if` dentro de `if` y cada vez que arregla un caso se rompe otro: la página no sabe en qué paso está, a veces pide el número 5 de 4 y, si se deja un campo vacío, vuelve al principio.

María le propone parar de escribir código y dibujar antes en un papel **qué páginas** puede ver el usuario, **qué botones** puede pulsar en cada una y **qué tiene que comprobar** el programa antes de pasar de una a otra. —Cuando lo tengas dibujado —le dice—, el código sale casi solo: es un `switch` y unos `if`. **Juan** sonríe: es la misma técnica con la que se programan los protocolos de comunicaciones, y funciona igual de bien con las páginas web.

En esta unidad aprenderás a:

- describir el comportamiento de una página PHP con un **diagrama de estado-transición ampliado**,
- identificar sus elementos: **eventos entrantes**, **eventos salientes**, **estados**, **variables de estado**, **predicados** y **acciones**,
- formalizarlo en un **autómata**: tabla de descriptores, tabla de eventos-estados y tabla de transiciones,
- comprobar que los predicados de cada transición cubren **todos los casos** y **no se solapan**,
- programar la página **a partir del autómata**, de forma casi mecánica.

La técnica viene de los libros de comunicaciones (Halsall, *Data Communications, Computer Networks and Open Systems*), donde se usa para especificar protocolos. En el módulo la usamos para resolver los ejercicios del tema 2. Ya hay tres resueltos así en los ejemplos del repositorio (Boletín 01-E01, Boletín 03-E01 y la Tarea 02A); con estos apuntes vienen diez más: cuatro problemas de los boletines y de la Tarea 02B, el Boletín 01-E01 rehecho y completado, y cinco problemas nuevos con más estados y transiciones (cajero, máquina expendedora, reservas, tres en raya y examen tipo test).

1.- Por qué un autómata para una página web

Caso práctico

Carlos se da cuenta de que su problema no es PHP: es que no tiene claro cuántas "situaciones" distintas tiene su página y qué hace en cada una.

Una página PHP que se usa varias veces seguidas (un formulario que se corrige, un juego, un asistente en varios pasos) tiene un problema de fondo: **HTTP no tiene memoria**. Cada vez que el usuario pulsa un botón, el navegador hace una petición nueva y el script empieza desde cero: no sabe qué pasó en la petición anterior si no se lo dice la propia página (en campos ocultos, en la URL... o, en el tema 5, en la sesión).

Por eso, cada petición se puede ver como un **evento** que llega a una página que está en un **estado**. Según el evento y el estado, y según se cumplan o no ciertas **condiciones**, el programa hace unas **acciones**, muestra una página (un **evento saliente**) y pasa a otro estado. Eso es exactamente un autómata de estado-transición.

> **Debes conocer**
>
> El estado no vive en el servidor: **viaja con la página**. Si el autómata tiene más de un estado, la página debe enviar en un campo oculto en qué estado está (`<input type="hidden" name="estado" value="2">`), y todo lo que haga falta recordar (contadores, sumas, tableros...). Y como el usuario puede cambiar esos campos con las herramientas del navegador, el programa debe **comprobar** el estado y los datos ocultos que recibe.

2.- Los elementos del diagrama



Figura 1. Cómo se lee una flecha del diagrama de estado-transición ampliado

Elemento	Abreviatura	Qué es en una página PHP	Ejemplos
Evento entrante	EE0, EE1...	lo que provoca una petición: cargar la página, pulsar un botón, pulsar un enlace	Enviar, Borrar, Carga, Jugada
Evento saliente	ES0, ES1...	lo que se muestra al usuario como respuesta (cada variante distinta de la página)	formulario vacío, formulario con errores, resultados
Estado	ST0, ST1...	la situación en la que está la página: qué formulario muestra y qué espera	Carga de página, Esperando datos, Jugando
Variable de estado	VE0, VE1...	lo que hay que recordar entre peticiones (viaja en campos ocultos)	`$estado`, `$contador`, `$sumatorio`, `$tablero`
Predicado lógico	PL0, PL1...	una condición que se evalúa a `true` o `false`; elige el camino	"el importe es un número mayor que 0"
Acción específica	AC0, AC1...	una instrucción (o un grupo) de PHP	`$suma = $a + $b`, "limpiar y validar el nombre"
Transición	T1, T2...	lo que pasa al llegar un evento en un estado	ver la tabla de transiciones

Algunos convenios que usamos en los diagramas:

- **STx** (círculo de borde grueso) representa *cualquier estado*: una flecha que sale de STx (por ejemplo, con el evento "Borrar" o "Carga") significa que ese evento hace lo mismo esté la página en el estado que esté.

- **ST0, Carga de página** es el estado de partida: la petición llega sin campo `estado` (la primera vez, o al pulsar un enlace a la página sin parámetros). Desde ST0 siempre se muestra la página inicial y se pasa a ST1.
- Las acciones que se hacen **antes** de comprobar los predicados (leer `$_GET`, limpiar y validar) se escriben en gris con "antes:"; las que se hacen **después**, en verde. Esta distinción es la de la Tarea 02A: "AEs PRE PLs" y "AEs POST PLs".
- En los diagramas grandes, una **caja discontinua** con varios estados ("ST3, ST4") es como STx pero solo para los estados que nombra: la flecha que sale de ella vale para todos. Sirve para no llenar el dibujo de flechas iguales (*Salir*, *Cancelar*, *Recoger*...); la tabla de eventos-estados sigue teniendo cada celda.
- Un **evento interno** no viene del navegador: lo produce el propio programa. Por ejemplo, en el tres en raya, tras la jugada del jugador el autómata pasa al estado "turno de la máquina" y, en la misma petición, se atiende el evento interno que hace la jugada del servidor. Ese estado es **transitorio**: nunca se envía a la página.
- Un evento saliente con acciones entre corchetes, `ES3 [AC4]`, significa que al generar esa página se ejecuta esa acción (normalmente, escribir las variables de estado en campos ocultos).

3.- El método, paso a paso

Caso práctico

María le enseña a Carlos el orden en que ella rellena las tablas. —No intentes dibujar el diagrama a la primera: primero haz listas.

- **Lee el enunciado y dibuja la secuencia de páginas** que verá el usuario en un uso normal (como hace el boletín con "la misma página en distintos estados progresivos"). Añade los casos de error: campo vacío, número fuera de rango...
- **Eventos entrantes (EE).** Haz una lista de todo lo que puede provocar una petición: cada botón `submit`, cada enlace, la primera carga. Los botones `type="reset"` no llegan al servidor: los resuelve el navegador (se anotan como *HTML/JS*).
- **Eventos salientes (ES).** Una entrada por cada variante distinta de la página: inicial, con errores, con resultados, final...
- **Estados (ST).** Un estado por cada situación en la que la página espera algo distinto. Pregunta clave: *¿necesito saber algo de la petición anterior para atender esta?* Si la respuesta no cambia, es el mismo estado.
- **Variables de estado (VE).** Lo que debe sobrevivir de una petición a la siguiente. Cada una tendrá su campo oculto (y su validación al volver).
- **Predicados (PL).** Cada condición **atómica**: "el lado es impar", "la casilla esconde una letra". Mejor muchos predicados simples que pocos complicados: así se pueden combinar y comprobar.
- **Acciones (AC).** Lo que hay que calcular o cambiar. Distingue las previas (validar las entradas, que se necesitan para evaluar los predicados) de las que se hacen según el camino elegido.
- **Dibuja el diagrama**: un círculo por estado y una flecha por cada camino posible, con su evento, sus predicados, sus acciones y su evento saliente.
- **Tabla de eventos-estados**: una fila por estado y una columna por evento entrante; en cada celda, el número de la transición (o `---` si ese evento no puede darse en ese estado, o `HTML/JS`).

- **Tabla de transiciones**: para cada número, sus caminos: acciones previas, predicados, nuevo estado, acciones y eventos salientes.
- **Comprueba los predicados** de cada transición (apartado 4).
- **Programa** siguiendo el autómata (apartado 5) y **prueba un camino por cada fila** de la tabla de transiciones.

4.- Comprobar los predicados: completos y excluyentes

Cuando una transición tiene varios caminos, sus predicados deben cumplir dos reglas:

- **Completos**: para cualquier combinación de valores de los predicados, **algún** camino se cumple. Si no, hay un caso que el programa no sabe tratar (y la página se queda en blanco o hace algo raro).
- **Excluyentes**: para cada combinación, se cumple **uno solo**. Si se cumplen dos, el resultado depende del orden de los `if`, y eso suele ser un error escondido.

Se comprueba con una **tabla de verdad**: una fila por cada combinación de los predicados (2 predicados, 4 filas; 3 predicados, 8 filas...) y una columna por camino. En cada fila debe haber **exactamente un 1**. Por ejemplo, para la T2 del Boletín 01-E01:

PL0	PL1	`!PL0 || !PL1`	`PL0 && PL1`
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

Cuando un predicado solo tiene sentido si otros se cumplen (por ejemplo, "el día existe en ese mes" solo se puede comprobar si el mes es válido), en la tabla se escribe **x** ("da igual, no se evalúa") y en el código va **detrás** en un `&&`: PHP evalúa en cortocircuito y no llega a calcularlo.

> **Recomendación**
>
> Haz siempre esta comprobación antes de programar. En el Boletín 01-E01 del material, por ejemplo, falta un predicado: si el segundo número es 0, `$numero1 % $numero2` lanza `DivisionByZeroError` en PHP 8. Con la tabla de predicados delante es fácil verlo: el resto solo se puede calcular si se cumple "el segundo número no es 0" (el hueco PL2 que quedó en blanco en la tabla de los ejemplos del repositorio).

5.- Del autómata al código

El código tiene siempre la misma forma: leer el estado y el evento, consultar la tabla y ejecutar la transición. Esta es la plantilla que siguen las soluciones de los cuatro ejemplos (la misma idea que el `index.php` del Boletín 01-E01 de los ejemplos del repositorio):

```php
<?php
// Estados y eventos entrantes: constantes con el nombre del autómata
const ST0 = 0;  const ST1 = 1;          // Carga de página, Esperando datos
const EE0 = 0;  const EE1 = 1;          // Calcular, Borrar

// Tabla de eventos-estados ampliada: $automata[estado][evento] = número de transición
$automata = [
    ST0 => [EE0 => 1, EE1 => 1],
    ST1 => [EE0 => 2, EE1 => 1],
];

// 1. Estado (campo oculto, VALIDADO) y evento entrante (qué botón ha llegado)
$estado = filter_input(INPUT_GET, 'estado', FILTER_VALIDATE_INT);
if ($estado === null || $estado === false || !isset($automata[$estado])) {
    $estado = ST0;                      // primera carga o estado manipulado: empezar
}
$evento = ($_GET['evento'] ?? '') === 'calcular' ? EE0 : EE1;

// 2. La transición que dice la tabla
switch ($automata[$estado][$evento] ?? 1) {     // ?? 1: evento imposible (---) -> empezar
    case 1:                                     // ---- T1
        $estado = ST1;
        es0();                                  // evento saliente: una función por ES
        break;
    case 2:                                     // ---- T2
        // acciones previas (AEs PRE): leer, limpiar y validar
        $pl0 = ...;                             // predicados como variables booleanas
        $pl1 = ...;
        if (!$pl0 || !$pl1) {                   // un if / elseif por camino, en el orden de
la tabla
            $estado = ST1;
            es1(...);
        } else {                                // PL0 && PL1
            $estado = ST1;
            // acciones (AEs POST)
            es2(...);
        }
        break;
}
// 3. El nuevo estado viaja con la página
echo '<input type="hidden" name="estado" value="' . $estado . '">';
```

Observa:

- Las abreviaturas del autómata aparecen en el código como **constantes y comentarios** (`ST1`, `// T2`, `// PL0 && PL1`, `// AC3`): quien lea el código puede ir del diagrama al programa y al revés.
- Cada **evento saliente** es una función (`es0()`, `es1()`...): así queda claro qué página se muestra en cada camino.

- Los predicados se calculan en **variables booleanas** con el nombre del predicado (`$pl0`); el `if` queda casi igual que en la tabla.
- La celda `---` de la tabla no puede darse usando la página normalmente (no hay botón para ese evento). Si llega, alguien ha manipulado la página: lo más seguro es **empezar de nuevo** (T1).

En los ejemplos del repositorio hay también una versión más "pura" del Boletín 03-E01 (no incluida en este material) en la que las transiciones son **datos**: un array con, para cada transición, el predicado, el nuevo estado, las acciones y la salida. Es una buena forma de ver que el autómata es una tabla; la versión con `switch` es más fácil de depurar paso a paso.

6.- Errores frecuentes

Error	Consecuencia	Cómo evitarlo
No guardar el estado en la página	la página no sabe en qué paso está y vuelve al principio	campo oculto `estado` en todos los formularios
Fiarse del estado y de los ocultos	un usuario los cambia con F12 y llega a un estado imposible	validarlos como cualquier otra entrada; si no valen, T1
Predicados que no cubren todos los casos	combinaciones en las que no se muestra nada	tabla de verdad: un 1 en cada fila
Predicados que se solapan	el resultado depende del orden de los `if`	tabla de verdad: un solo 1 en cada fila
Evaluar un predicado que no tiene sentido	avisos o errores (dividir entre 0, índice que no existe)	ponerlo detrás de los que lo hacen posible en un `&&`
Mezclar estados con eventos salientes	demasiados estados, todos iguales	un estado nuevo solo si la página espera algo distinto
Olvidar los eventos de "Borrar" o de recarga	el usuario no puede empezar de nuevo	columna para cada botón y para la carga sin parámetros

7.- Ejemplos resueltos

Ejemplo	Qué practica	Estados	Eventos	Dónde está
Boletín 01-E01: suma, resta, producto y resto	lo mínimo: dos estados y un formulario	2	Enviar, Reset	ejemplos del repositorio; **rehecho y completado** (PL2, el cociente) en `05-B01-E01-Operaciones`
Boletín 03-E01: media de una secuencia	un contador en campos ocultos; varios caminos	4	Enviar, Borrar	ejemplos del repositorio
Tarea 02A: agenda	un array en campos ocultos; acciones antes y después de los predicados	3	Enviar, Limpiar, Vaciar	ejemplos del repositorio; su autómata, en `EJEMPLO-TAREA-02A-Agenda`
Boletín 02-E01: impuestos	validación de dos campos, acciones previas	2	Calcular, Borrar	`01-B02-E01-Impuestos`
Boletín 03-E08: día de la semana	cuatro predicados, uno que depende de los otros (cortocircuito); `type="reset"`	2	Carga, Enviar, (Borrar)	`02-B03-E08-DiaSemana`
Boletín 05-E04: producto de matrices	dos formularios seguidos, matrices en campos ocultos, eventos imposibles	4	Carga, Enviar	`03-B05-E04-ProductoMatrices`
Tarea 02B: juego de tablero	cinco estados, tres eventos, caminos imposibles en un estado	5	Carga, Enviar, Jugada	`EJEMPLO-TAREA-02B-JuegoTablero`
Cajero automático	la cuenta en campos ocultos **firmados** (`hash_hmac`); PIN con intentos; cascadas de predicados	8	9	`06-Cajero`
Máquina expendedora	datos en el valor del botón; un predicado que **busca** (dar el cambio con las monedas que hay); modo mantenimiento	7	9	`07-Expendedora`
Reservas del Bar Baio	asistente con *Atrás* y *Editar* desde el resumen: una variable de estado (`$editando`) cambia el destino; fechas	7	6	`08-ReservaBarBaio`
Tres en raya contra el servidor	**evento interno** (el turno de la máquina) y estado transitorio	7	7	`09-TresEnRaya`
Examen tipo test	preguntas saltadas, revisión de pendientes, dos modos; predicados evaluados después de una acción	6	8	`10-ExamenTest`

Para empezar, el más sencillo es el Boletín 01-E01. Cada ejemplo nuevo tiene su documento con el enunciado, el análisis, el diagrama, las tablas del autómata, la comprobación de los predicados, la

solución en PHP programada a partir del autómata (`solucion/index.php`, dentro de la carpeta de cada ejemplo en `../02-Ejemplos/`) y las pruebas que recorren todos los caminos.

Autoevaluación

- ¿Por qué una página PHP con varios pasos necesita un campo oculto con el estado? **Porque HTTP no tiene memoria: cada petición empieza desde cero y la única forma de saber en qué paso está la página es que la propia página lo envíe.**
- Un botón `<button type="reset">`, ¿es un evento entrante del autómata del servidor? **No: lo resuelve el navegador (borra los campos) y no llega ninguna petición al servidor. En la tabla de eventos-estados se anota como HTML/JS.**
- Una transición tiene los caminos `PL0 && PL1` y `!PL0`. ¿Están bien? **No: con PL0 cierto y PL1 falso no se cumple ninguno (no son completos). Falta el camino `PL0 && !PL1`.**
- ¿Qué significa `---` en una celda de la tabla de eventos-estados? **Que ese evento no puede ocurrir en ese estado usando la página normalmente (no hay botón para él). Si llega, la página se ha manipulado y lo prudente es empezar de nuevo.**
- ¿Por qué "el día existe en ese mes" se escribe detrás de "el mes es válido" en el `&&`? **Porque solo se puede calcular si el mes es válido; el `&&` de PHP no evalúa la segunda parte si la primera es falsa (cortocircuito).**
